EC API Documentation

Embedded Controller - STM32F411 Driver Library

Written by: Juyoung Lee

Program: C/C++

IDE/Compiler: Keil uVision 5

OS: WIn10/11

MCU: STM32F411RE, Nucleo-64

Table of Contents

EC API Documentation

- Embedded Controller - STM32F411 Driver Library
- 1 Pin Names & Bitwise Macros (`ecPinNames.h` / `ecPinNames.c`)
  - Header File (`ecPinNames.h`)
  - Functions & Bitwise Macros
    - Bitwise Helper Macros
    - `ecPinmap()`
- 2 GPIO Digital InOut
    - Header File
  - 2.1 GPIO
    - GPIO_init()
    - GPIO_mode()
    - GPIO_write()
    - GPIO_read()
    - GPIO_ospeed()
    - GPIO_otype()
    - GPIO_pupd()
    - Example Code
  - 2.2 Arduino Syntax
    - pinMode()
    - digitalWrite()
    - digitalRead()
    - Example Code
  - 2.3 FND for JKIT board
    - FND_display_init()
    - FND_select_init()
    - FND_display()
    - FND_select()
    - Example Code
- 3 EXTI
  - Header File
    - `EXTI_init()`
    - `EXTI_enable()`
    - `EXTI_disable()`
    - `is_pending_EXTI()`
    - `clear_pending_EXTI()`
    - **Example code**

- 4 Stepper Motor
  - Stepper Motor Functions
    - `stepper_init()`
    - `stepper_speed()`
    - `stepper_step()`
    - `stepper_out()`
    - `stepper_stop()`
    - Complete Example Code (Stepper Motor Control)
- 5 System Clock (`ecRCC2.h` / `ecRCC2.c`)
  - Header File (`ecRCC2.h`)
  - RCC Functions
    - `RCC_HSI_init()`
    - `RCC_PLL_init()`
    - `delay_ms_HSI()`
    - `RCC_GPIOx_enable()`
- 6 SysTick Timer (`ecSysTick2.h` / `ecSysTick2.c`)
  - Header File (`ecSysTick2.h`)
  - SysTick Functions
    - `SysTick_init()`
    - `delay_ms()`
    - `SysTick_reset()`
    - `SysTick_val()`
- 7 STM32 Board Initialization Helper (`ecSTM32F4v2.h`)
  - Header File (`ecSTM32F4v2.h`)
  - Initialization Functions
    - `MCU_init()`
    - `MCU_JK_init()`

---

1 Pin Names & Bitwise Macros (`ecPinNames.h` / `ecPinNames.c`)

Header File (`ecPinNames.h`)

```
#include "stm32f411xe.h"

// Bitwise Macro Definition
#define BIT_SET(REG, BIT)          ((REG) |= 1 << (BIT))
#define BIT_CLEAR(REG, BIT)        ((REG) &= ~(1 << (BIT)))
#define BIT_READ(REG, BIT)         (((REG) >> (BIT)) & 1)
#define BITS_SET(REG, BIT, NUM)    ((REG) |= (NUM) << (BIT))
#define BITS_CLEAR(REG, BIT, NUM)  ((REG) &= ~((NUM) << (BIT)))

// Pinname Config
typedef enum {
    PortA = 0, PortB = 1, PortC = 2, PortD = 3,
    PortE = 4, PortF = 5, PortG = 6, PortH = 7,
    PortI = 8, PortJ = 9, PortK = 10
} PortName_t;
```

```
typedef enum {
    PA_0  = 0x00, PA_1  = 0x01, PA_2  = 0x02, PA_3  = 0x03,
    PA_4  = 0x04, PA_5  = 0x05, PA_6  = 0x06, PA_7  = 0x07,
    PA_8  = 0x08, PA_9  = 0x09, PA_10 = 0x0A, PA_11 = 0x0B,
    PA_12 = 0x0C, PA_13 = 0x0D, PA_14 = 0x0E, PA_15 = 0x0F,

    PB_0  = 0x10, PB_1  = 0x11, PB_2  = 0x12, PB_3  = 0x13,
    PB_4  = 0x14, PB_5  = 0x15, PB_6  = 0x16, PB_7  = 0x17,
    PB_8  = 0x18, PB_9  = 0x19, PB_10 = 0x1A, PB_12 = 0x1C,
    PB_13 = 0x1D, PB_14 = 0x1E, PB_15 = 0x1F,

    PC_0  = 0x20, PC_1  = 0x21, PC_2  = 0x22, PC_3  = 0x23,
    PC_4  = 0x24, PC_5  = 0x25, PC_6  = 0x26, PC_7  = 0x27,
    PC_8  = 0x28, PC_9  = 0x29, PC_10 = 0x2A, PC_11 = 0x2B,
    PC_12 = 0x2C, PC_13 = 0x2D, PC_14 = 0x2E, PC_15 = 0x2F,

    PD_2  = 0x32, PH_0  = 0x70, PH_1  = 0x71,

    // Arduino connector namings
    A0 = PA_0, A1 = PA_1, A2 = PA_4, A3 = PB_0, A4 = PC_1, A5 = PC_0,
    D0 = PA_3, D1 = PA_2, D2 = PA_10, D3 = PB_3, D4 = PB_5, D5 = PB_4,
    D6 = PB_10, D7 = PA_8, D8 = PA_9, D9 = PC_7, D10 = PB_6, D11 = PA_7,
    D12 = PA_6, D13 = PA_5, D14 = PB_9, D15 = PB_8,

    NC = (int)0xFFFFFFFF
} PinName_t;

void ecPinmap(PinName_t pinName, GPIO_TypeDef **GPIOx, unsigned int *pin);
```

Functions & Bitwise Macros

Bitwise Helper Macros

- `BIT_SET(REG, BIT)`: Sets the bit at position `BIT` in register `REG`.
- `BIT_CLEAR(REG, BIT)`: Clears the bit at position `BIT` in register `REG`.
- `BIT_READ(REG, BIT)`: Reads and returns the binary value (`0` or `1`) at position `BIT` in register `REG`.
- `BITS_SET(REG, BIT, NUM)`: Writes a multi-bit mask `NUM` starting at position `BIT`.
- `BITS_CLEAR(REG, BIT, NUM)`: Clears a multi-bit mask `NUM` starting at position `BIT`.

`ecPinmap()`

Extracts the GPIO register peripheral pointer (`GPIOx`) and bit position (`pin`) corresponding to a given `PinName_t` alias.

```
void ecPinmap(PinName_t pinName, GPIO_TypeDef **GPIOx, unsigned int *pin);
```

- **Parameters:**
  - `pinName`: Pin identifier of type `PinName_t` (e.g., `PA_5`, `D13`)

    - `GPIOx`: Pointer to `GPIO_TypeDef*` where the GPIO port address (`GPIOA`, `GPIOB`, etc.) will be stored
    - `pin`: Pointer to an `unsigned int` where the bit index (`0` ~ `15`) will be stored
- **Example Code:**

```
GPIO_TypeDef *port;
unsigned int pinNo;
ecPinmap(PA_5, &port, &pinNo); // Sets port = GPIOA, pinNo = 5
```

2 GPIO Digital InOut

Header File

`#include "ecGPIO.h"`

```
// MODER
#define INPUT       0x00
#define OUTPUT      0x01
#define AF          0x02
#define ANALOG      0x03

// IDR & ODR
#define HIGH        0x01
#define LOW         0x00

// OSPEED
typedef enum {
    SPEED_LOW         = 0,
    SPEED_MEDIUM      = 1,
    SPEED_FAST        = 2,
    SPEED_HIGH        = 3,
}OSPEED;

// OTYPE
typedef enum {
    PUSH_PULL         = 0,
    OPEN_DRAIN        = 1,
} OTYPE;

// PUPD
typedef enum {
    NO_PUPD           = 0,
    PULL_UP           = 1,
    PULL_DOWN         = 2,
    RESERVED          = 3,
}PUPD;


// GPIO
```

```
void GPIO_init      (PinName_t pinName, uint32_t mode);
void GPIO_mode      (PinName_t pinName, uint32_t mode);
void GPIO_ospeed    (PinName_t pinName, uint32_t speed);
void GPIO_otype     (PinName_t pinName, uint32_t type);
void GPIO_pupd      (PinName_t pinName, uint32_t pupd);
void GPIO_write     (PinName_t pinName, uint32_t Output);
uint32_t  GPIO_read (PinName_t pinName);

// ARDUINO Syntax
void        pinMode             (PinName_t pin, uint32_t mode);
void        digitalWrite        (PinName_t pin, uint32_t value);
uint32_t    digitalRead         (PinName_t pin);

// FND for JKIT board
extern int numberFND[12][8];
void FND_display_init       (PinName_t *pinFND);
void FND_select_init        (PinName_t *selectFND);
void FND_display            (uint8_t num);
void FND_select             (uint8_t digit);
```

2.1 GPIO

GPIO_init()

Initializes GPIO pins with default setting and Enables GPIO Clock. Mode: In/Out/AF/Analog

```
void GPIO_init(PinName_t pinName, uint32_t mode);
```

Parameters

- **pinName:** (port name)_(pin number) (PinName_t) PA_0
- **mode**: INPUT(0), OUTPUT(1), AF(02), ANALOG (03)

Example code

```
GPIO_init(PA_0, INPUT);
```

GPIO_mode()

Configures GPIO pin modes: In/Out/AF/Analog

```
void GPIO_mode(PinName_t pinName, uint32_t mode);
```

Parameters

- **pinName:** (port name)_(pin number) (PinName_t) PA_0
- **mode**: INPUT (0), OUTPUT (1), AF(02), ANALOG (03)

Example code

```
GPIO_mode(PA_0, INPUT);
```

GPIO_write()

Write the data to GPIO pin: High, Low

```
void GPIO_write(PinName_t pinName, uint32_t Output);
```

Parameters

- **pinName:** (port name)_(pin number) (PinName_t) PB_12
- **output**: LOW(0), HIGH(1)

Example code

```
GPIO_write(PB_12, HIGH);
```

GPIO_read()

Read the data from GPIO pin

```
uint32_t  GPIO_read(PinName_t pinName);
```

Parameters

- **pinName:** (port name)_(pin number) (PinName_t) PA_0

Example code

```
GPIO_read(PA_0);
```

GPIO_ospeed()

Configures output speed of GPIO pin : Low, Mid, Fast, High

```
void GPIO_ospeed(PinName_t pinName, uint32_t speed);
```

Parameters

- **pinName:** (port name)_(pin number) (PinName_t) PB_12
- **speed**: SPEED_LOW (0), SPEED_MEDIUM (1), SPEED_FAST (2) , SPEED_HIGH (3)

Example code

```
GPIO_ospeed(PB_12, SPEED_MEDIUM);
```

GPIO_otype()

Configures output type of GPIO pin: Push-Pull / Open-Drain

```
void GPIO_otype(PinName_t pinName, uint32_t type);
```

Parameters

- **pinName:** (port name)_(pin number) (PinName_t) PB_12
- **type**: PUSH_PULL(0), OPEN_DRAIN(1)

Example code

```
GPIO_otype(PB_12, PUSH_PULL);
```

GPIO_pupd()

Configures Pull-up/Pull-down mode of GPIO pin: No Pull-up, Pull-down/ Pull-up/ Pull-down/ Reserved

```
void GPIO_pupd(PinName_t pinName, uint32_t pupd);
```

Parameters

- **pinName:** (port name)_(pin number) (PinName_t) PB_12
- **pupd**: NO_PUPD(0), PULL_UP(1), PULL_DOWN(2), RESERVED(3)

Example code

```
GPIO_pupd(PB_12, PULL_UP);
```

Example Code

```
#include "ecRCC2.h"
#include "ecGPIO2.h"

#define LED_PIN PB_12
#define BUTTON_PIN PA_0

// Initialiization
void setup(void) {
    RCC_HSI_init();

    // initialize the pushbutton pin as an input:
    GPIO_init(BUTTON_PIN, INPUT);
    GPIO_pupd(BUTTON_PIN, PULL_UP);

    // initialize the LED pin as an output:
    GPIO_init(LED_PIN, OUTPUT);
    GPIO_pupd(LED_PIN, PULL_UP);
    GPIO_otype(LED_PIN, PUSH_PULL);
    GPIO_ospeed(LED_PIN, SPEED_MEDIUM);

}

int main(void) {
```

```
    setup();
    int buttonState=0;

    while(1){
        // check if the pushbutton is pressed. Turn LED on/off accordingly:
        buttonState = GPIO_read(BUTTON_PIN);
        if(buttonState) GPIO_write(LED_PIN, LOW);
        else GPIO_write(LED_PIN, HIGH);
        delay_ms_HSI(100);                              // delay 100 ms
    }
}
```

2.2 Arduino Syntax

pinMode()

Configures GPIO pin modes: In/Out/AF/Analog

```
void pinMode(PinName_t pin, uint32_t mode);
```

Parameters

- **pinName:** (port name)_(pin number) (PinName_t) PA_0
- **mode**: INPUT (0), OUTPUT (1), AF(02), ANALOG (03)

Example code

```
pinMode(PB_12, OUTPUT)
```

digitalWrite()

Write the data to GPIO pin: High, Low

```
void digitalWrite(PinName_t pin, uint32_t value);
```

Parameters

- **pinName:** (port name)_(pin number) (PinName_t) PB_12
- **output**: LOW(0), HIGH(1)

Example code

```
digitalWrite(PB_12, HIGH);
```

digitalRead()

Read the data from GPIO pin

```
uint32_t digitalRead(PinName_t pin);
```

Parameters

- **pinName:** (port name)_(pin number) (PinName_t) PB_12

Example code

```
digitalRead(PA_0);
```

Example Code

```
#include "ecRCC2.h"
#include "ecGPIO2.h"

#define LED_PIN    PB_12
#define BUTTON_PIN PA_0

void setup(void) {
    RCC_HSI_init();

    // Configure pins using Arduino Syntax
    pinMode(BUTTON_PIN, INPUT);
    GPIO_pupd(BUTTON_PIN, PULL_UP); // Pull-up configuration for active-low button

    pinMode(LED_PIN, OUTPUT);
    GPIO_pupd(LED_PIN, PULL_UP);
    GPIO_otype(LED_PIN, PUSH_PULL);
    GPIO_ospeed(LED_PIN, SPEED_MEDIUM);
}

int main(void) {
    setup();

    while(1) {
        // Read input pin and update LED state using Arduino API
        uint32_t buttonState = digitalRead(BUTTON_PIN);

        if (buttonState == LOW) { // Button pressed
            digitalWrite(LED_PIN, HIGH);
        } else {                  // Button released
            digitalWrite(LED_PIN, LOW);
        }

        delay_ms_HSI(50); // Debounce delay
    }
}
```

2.3 FND for JKIT board

FND_display_init()

Initializes and configures the 8 GPIO segment pins (a, b, c, d, e, f, g, dp) as output pins.

```
void FND_display_init(PinName_t *pinFND);
```

- **Parameters:**
  - `pinFND`: Pointer to an array of 8 `PinName_t` values specifying the segment pins `[a, b, c, d, e, f, g, dp]`.
- **Example Code:**

```
PinName_t fndPins[8] = {PA_5, PA_6, PA_7, PB_6, PC_7, PA_9, PA_8, PB_10};
FND_display_init(fndPins);
```

FND_select_init()

Initializes and configures the digit selection GPIO pins as output pins for multi-digit display control.

```
void FND_select_init(PinName_t *selectFND);
```

- **Parameters:**
  - `selectFND`: Pointer to an array of `PinName_t` values specifying the digit select pins (e.g., Digit 1 to Digit 4).
- **Example Code:**

```
PinName_t selectPins[4] = {PC_6, PC_8, PC_9, PB_8};
FND_select_init(selectPins);
```

FND_display()

Outputs the specified numeric digit pattern onto the currently selected segment display.

```
void FND_display(uint8_t num);
```

- **Parameters:**
  - `num`: Integer value to display (`0` ~ `11`). `0`–`9` render numbers 0 to 9, while `10`–`11` correspond to special characters or symbols defined in `numberFND`.
- **Example Code:**

```
FND_display(7); // Displays number 7 on active digit
```

FND_select()

Activates a specific digit line for multiplexed multi-digit FND driving.

```
void FND_select(uint8_t digit);
```

- **Parameters:**
  - `digit`: Digit index to activate (e.g., `0` for 1st digit, `1` for 2nd digit, `2` for 3rd digit, `3` for 4th

digit).

- **Example Code:**

```
FND_select(0); // Activates the 1st digit position
```

Example Code

```
#include "ecRCC2.h"
#include "ecGPIO2.h"

// Define FND segment pins [a, b, c, d, e, f, g, dp]
PinName_t fndPins[8] = {PA_5, PA_6, PA_7, PB_6, PC_7, PA_9, PA_8, PB_10};

// Define FND digit select pins [DIG0, DIG1, DIG2, DIG3]
PinName_t selectPins[4] = {PC_6, PC_8, PC_9, PB_8};

void setup(void) {
    RCC_HSI_init();

    // Initialize FND segment and select pins
    FND_display_init(fndPins);
    FND_select_init(selectPins);
}

int main(void) {
    setup();

    while(1) {
        // Display '1' on Digit 0
        FND_select(0);
        FND_display(1);
        delay_ms_HSI(5);

        // Display '2' on Digit 1
        FND_select(1);
        FND_display(2);
        delay_ms_HSI(5);
    }
}
```

---

3 EXTI

Header File

`#include ecEXTI2.h`

```
// Trigger Type Definitions
#define FALL 0
#define RISE 1
#define BOTH 2

void EXTI_init(PinName_t pinName, uint32_t trig_type, uint32_t priority);
void EXTI_enable(PinName_t pinName);
void EXTI_disable(PinName_t pinName);
uint32_t is_pending_EXTI(PinName_t pinName);
void clear_pending_EXTI(PinName_t pinName);
```

`EXTI_init()`

Initializes and configures external interrupt for the specified GPIO pin line, including trigger edge selection and NVIC priority setting.

```
void EXTI_init(PinName_t pinName, uint32_t trig_type, uint32_t priority);
```

- **Parameters:**
  - `pinName`: `(port name)_(pin number)` of type `PinName_t` (e.g., `PA_0`, `PC_13`)
  - `trig_type`: Trigger condition — `FALL` (`0`), `RISE` (`1`), `BOTH` (`2`)
  - `priority`: Interrupt priority level in NVIC (e.g., `0` ~ `15`)
- **Example Code:**

  ```
  EXTI_init(PA_0, FALL, 0); // Active-low button interrupt with highest priority
  ```

`EXTI_enable()`

Enables external interrupt mask (IMR) for the specified pin line.

```
void EXTI_enable(PinName_t pinName);
```

- **Parameters:**
  - `pinName`: `PinName_t` (e.g., `PA_0`)
- **Example Code:**

  ```
  EXTI_enable(PA_0);
  ```

`EXTI_disable()`

Disables external interrupt mask (IMR) for the specified pin line.

```
void EXTI_disable(PinName_t pinName);
```

- **Parameters:**
  - `pinName`: `PinName_t` (e.g., `PA_0`)

- **Example Code:**

```
EXTI_disable(PA_0);
```

`is_pending_EXTI()`

Checks if the EXTI pending bit for the specified pin line is set.

```
uint32_t is_pending_EXTI(PinName_t pinName);
```

- **Parameters:**
  - `pinName`: `PinName_t` (e.g., `PA_0`)
- **Return Value:** `1` if interrupt request is pending, `0` otherwise.
- **Example Code:**

```
if (is_pending_EXTI(PA_0)) {
    // Handle interrupt event
}
```

`clear_pending_EXTI()`

Clears the pending bit in the EXTI pending register (PR) for the specified pin line.

```
void clear_pending_EXTI(PinName_t pinName);
```

- **Parameters:**
  - `pinName`: `PinName_t` (e.g., `PA_0`)
- **Example Code:**

```
clear_pending_EXTI(PA_0);
```

Example code

This example uses two switch inputs via EXTI to control a 2-digit 7-segment counter:

```
#include "ecSTM32F4v2.h"

extern volatile uint32_t msTicks;

uint8_t numDisplay1 = 0;
uint8_t numDisplay2 = 0;
uint8_t selectFND1  = 0;
uint8_t selectFND2  = 1;

uint8_t state = 0;
uint8_t pause = 0;

volatile uint32_t curTicks = 0;
```

```c
// Initialization
void setup(void) {
    MCU_JK_init();
    curTicks = msTicks;
}

// MAIN -----------------------------------------
int main(void) {
    setup();

    while (1) {
        // 1) Write - Decompose state into digits and multiplex display
        numDisplay1 = state % 10;
        numDisplay2 = state / 10;

        FND_select(selectFND1);
        FND_display(numDisplay1);
        delay_ms(5);

        FND_select(selectFND2);
        FND_display(numDisplay2);
        delay_ms(5);

        // 2) Update counter periodically when not paused
        if (((msTicks - curTicks) > EXTI_TIME_PERIOD) && (pause == 0)) {
            state = (state + 1) % EXTI_NUM_STATE;
            curTicks = msTicks;
        }
    }
}

// EXTI Handler for SW2: Toggle Pause State
void EXTI4_IRQHandler(void) {
    if (is_pending_EXTI(SW2_PIN_JK)) {
        pause ^= 1;          // Toggle pause flag
        curTicks = msTicks;  // Reset time reference

        clear_pending_EXTI(SW2_PIN_JK);
    }
}

// EXTI Handler for SW1: Reset State to #0
void EXTI2_IRQHandler(void) {
    if (is_pending_EXTI(SW1_PIN_JK)) {
        state = 0;           // Reset count value
        curTicks = msTicks;  // Reset time reference

        clear_pending_EXTI(SW1_PIN_JK);
    }
}
```

4 Stepper Motor

Header File (`ecStepper2.h`)

```
// Stepper Mode
#define HALF    0
#define FULL    1
#define DIR_CW  0
#define DIR_CCW 1

// Stepper Motor Spec (28BYJ-48 : 64 steps/rev x 1/32 gear = 2048 steps/rev)
#define STEP_PER_REV  (64*32)

// State number
typedef enum {
    S0, S1, S2, S3, S4, S5, S6, S7
} StateNum;

// FULL / HALF stepping sequence - FSM
typedef struct {
    uint32_t next[2];
    uint8_t out[4];
} StateStepper_t;

// Function Declarations
void stepper_init(uint32_t mode, PinName_t *pinStepper);
void stepper_speed(uint32_t speedRPM);
void stepper_step(uint32_t steps, uint32_t direction);
void stepper_out(uint32_t state);
void stepper_stop(void);
```

Stepper Motor Functions

`stepper_init()`

Initializes the 4 control GPIO pins (A, B, A', B') as digital outputs and sets the operating FSM mode (`FULL` or `HALF` stepping).

```
void stepper_init(uint32_t mode, PinName_t *pinStepper);
```

- **Parameters:**
  - `mode`: Stepping mode — `FULL` (`1`), `HALF` (`0`)
  - `pinStepper`: Pointer to an array of 4 `PinName_t` pins `[A, B, A', B']`
- **Example Code:**

```
PinName_t stepperPins[4] = {PB_10, PB_4, PB_5, PB_3};
stepper_init(FULL, stepperPins);
```

`stepper_speed()`

Calculates step delay in milliseconds based on the desired rotational speed (RPM) and stepping mode.

```
void stepper_speed(uint32_t speedRPM);
```

- **Parameters:**
  - `speedRPM`: Target rotational speed in RPM (Revolutions Per Minute)
- **Example Code:**

```
stepper_speed(14); // Set motor speed to 14 RPM
```

`stepper_step()`

Rotates the stepper motor for a specified number of steps in the given direction using FSM transitions.

```
void stepper_step(uint32_t steps, uint32_t direction);
```

- **Parameters:**
  - `steps`: Number of steps to move (e.g., `2048` steps for 1 full revolution in `FULL` mode)
  - `direction`: Direction of rotation — `DIR_CW` (`0`), `DIR_CCW` (`1`)
- **Example Code:**

```
stepper_step(STEP_PER_REV, DIR_CW); // Rotate 1 full revolution clockwise
```

`stepper_out()`

Outputs digital logic values (`0` or `1`) to the 4 motor drive pins corresponding to the specified FSM state.

```
void stepper_out(uint32_t state);
```

- **Parameters:**
  - `state`: Current state index (`S0` ~ `S7`) in the active FSM array
- **Example Code:**

```
stepper_out(S0);
```

`stepper_stop()`

Stops motor rotation and sets all 4 driver output pins to low logic (`0`) to prevent motor overheating and power consumption.

```
void stepper_stop(void);
```

- **Example Code:**

```
stepper_stop();
```

Complete Example Code (Stepper Motor Control)

This example rotates a 28BYJ-48 stepper motor 1 revolution Clockwise (`DIR_CW`), pauses for 1 second, rotates 1 revolution Counter-Clockwise (`DIR_CCW`), and then stops the motor.

```
#include "ecSTM32F4v2.h"
#include "ecStepper2.h"

// Define Stepper Motor Pins [A, B, A', B']
PinName_t stepperPins[4] = {PB_10, PB_4, PB_5, PB_3};

void setup(void) {
    RCC_HSI_init();

    // Initialize Stepper Motor in Full Step mode
    stepper_init(FULL, stepperPins);

    // Set motor speed to 12 RPM
    stepper_speed(12);
}

int main(void) {
    setup();

    while(1) {
        // 1. Rotate 1 full turn Clockwise
        stepper_step(STEP_PER_REV, DIR_CW);
        delay_ms(1000);

        // 2. Rotate 1 full turn Counter-Clockwise
        stepper_step(STEP_PER_REV, DIR_CCW);
        delay_ms(1000);

        // 3. Stop motor drivers to prevent heating
        stepper_stop();

        while(1); // Hold execution
    }
}
```

5 System Clock (`ecRCC2.h` / `ecRCC2.c`)

Header File (`ecRCC2.h`)

```
#include "stm32f4xx.h"
#include "stm32f411xe.h"
#include "ecPinNames.h"

extern volatile int EC_SYSCLK; // Current system clock frequency [Hz]
```

```
void RCC_HSI_init(void);
void RCC_PLL_init(void);
void delay_ms_HSI(uint32_t ms);

void RCC_GPIOA_enable(void);
void RCC_GPIOB_enable(void);
void RCC_GPIOC_enable(void);
void RCC_GPIOD_enable(void);
void RCC_GPIOE_enable(void);
void RCC_GPIOH_enable(void);

#ifdef __cplusplus
}
#endif /* __cplusplus */

#endif // __EC_RCC2_H
```

RCC Functions

`RCC_HSI_init()`

Initializes and enables the internal High-Speed Oscillator (HSI, 16 MHz) as the system clock source. Sets `EC_SYSCLK = 16000000`.

```
void RCC_HSI_init(void);
```

- **Example Code:**

  ```
  RCC_HSI_init(); // Configure system clock to 16 MHz
  ```

`RCC_PLL_init()`

Configures Flash latency and PLL multipliers to set the system clock (`SYSCLK`) to 84 MHz using the 16 MHz HSI oscillator. Sets `EC_SYSCLK = 84000000`.

```
void RCC_PLL_init(void);
```

- **Example Code:**

  ```
  RCC_PLL_init(); // Set clock to 84 MHz
  ```

`delay_ms_HSI()`

Executes a busy-wait loop delay in milliseconds based on the 16 MHz HSI clock.

```
void delay_ms_HSI(uint32_t ms);
```

- **Parameters:**

    - `ms`: Delay duration in milliseconds
- **Example Code:**

```
delay_ms_HSI(100); // Delay 100 ms
```

`RCC_GPIOx_enable()`

Enables peripheral clock gate for the specified GPIO port in `RCC->AHB1ENR`.

```
void RCC_GPIOA_enable(void);
void RCC_GPIOB_enable(void);
void RCC_GPIOC_enable(void);
void RCC_GPIOD_enable(void);
void RCC_GPIOE_enable(void);
void RCC_GPIOH_enable(void);
```

- **Example Code:**

```
RCC_GPIOA_enable(); // Enable GPIOA clock
```

6 SysTick Timer (`ecSysTick2.h` / `ecSysTick2.c`)

Header File (`ecSysTick2.h`)

```
#ifndef __EC_SYSTICK2_H
#define __EC_SYSTICK2_H

#include "stm32f4xx.h"
#include "ecRCC2.h"
#include <stdint.1h>

#ifdef __cplusplus
extern "C" {
#endif /* __cplusplus */

extern volatile uint32_t msTicks;

void SysTick_init(void);
void SysTick_Handler(void);
void SysTick_counter(void);
void delay_ms(uint32_t msec);
void SysTick_reset(void);
uint32_t SysTick_val(void);

#ifdef __cplusplus
}
#endif /* __cplusplus */
```

```
#endif // __EC_SYSTICK2_H
```

SysTick Functions

`SysTick_init()`

Configures the ARM Cortex-M SysTick core hardware timer to trigger an interrupt every 1 ms using the active `EC_SYSCLK` frequency.

```
void SysTick_init(void);
```

- **Note:** Must be called **after** calling `RCC_HSI_init()` or `RCC_PLL_init()`.
- **Example Code:**

  ```
  RCC_PLL_init();
  SysTick_init(); // Initialize 1 ms timer ticks
  ```

`delay_ms()`

Blocking millisecond delay using the global `msTicks` variable. Uses unsigned arithmetic subtraction to safely handle tick counter overflow.

```
void delay_ms(uint32_t msec);
```

- **Parameters:**
  - `msec`: Delay time in milliseconds
- **Example Code:**

  ```
  delay_ms(500); // Pause execution for 500 ms
  ```

`SysTick_reset()`

Resets the current SysTick counter value register (`VAL`) back to 0.

```
void SysTick_reset(void);
```

`SysTick_val()`

Reads and returns the current raw value of the SysTick down-counter register (`VAL`).

```
uint32_t SysTick_val(void);
```

- **Return Value:** Raw counter value

7 STM32 Board Initialization Helper (`ecSTM32F4v2.h`)

Header File (`ecSTM32F4v2.h`)

```
#include "stm32f4xx.h"
#include "stm32f411xe.h"
#include "math.h"

#include "ecPinNames.h"
#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecEXTI2.h"
#include "ecSysTick2.h"
#include "ecStepper2.h"

#define SW1_PIN          PC_13
#define LED_PIN          PA_5

#define SW2_PIN_JK       PA_4
#define SW1_PIN_JK       PD_2

#define EXTI_NUM_STATE   10
#define EXTI_TIME_PREIOD 1000 // [ms]

static PinName_t pinFND[8]    = {PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7};
static PinName_t selectFND[4] = {PA_10, PA_11, PC_4, PC_3};

static inline void MCU_JK_init(void) {
    RCC_PLL_init();
    SysTick_init();

    GPIO_init(SW1_PIN_JK, INPUT);
    GPIO_pupd(SW1_PIN_JK, PULL_UP);

    GPIO_init(SW2_PIN_JK, INPUT);
    GPIO_pupd(SW2_PIN_JK, PULL_UP);

    FND_display_init(pinFND);
    FND_select_init(selectFND);

    EXTI_init(SW2_PIN_JK, FALL, 10);
    EXTI_init(SW1_PIN_JK, FALL, 10);
}

static inline void MCU_init(void) {
    RCC_PLL_init();
    SysTick_init();

    GPIO_init(PC_13, INPUT);
    GPIO_pupd(PC_13, PULL_DOWN);

    GPIO_init(PA_5, OUTPUT);
```

```
}

#endif // __EC_STM_H
```

Initialization Functions

`MCU_init()`

Performs default peripheral initializations for standard STM32 Nucleo-64 board setups:

1. Configures PLL system clock to 84 MHz.
2. Configures SysTick timer for 1 ms tick timing.
3. Configures onboard User Button (`PC_13`) with Pull-Down resistor.
4. Configures onboard Green LED (`PA_5`) as Digital Output.

```
static inline void MCU_init(void);
```

`MCU_JK_init()`

Performs default peripheral initializations for the JKIT evaluation board:

1. Configures PLL system clock to 84 MHz and SysTick for 1 ms ticks.
2. Initializes push buttons `SW1_PIN_JK` (`PD_2`) and `SW2_PIN_JK` (`PA_4`) as inputs with pull-up resistors.
3. Initializes 7-segment display segment pins (`pinFND`) and digit select pins (`selectFND`).
4. Configures EXTI interrupt lines on `SW1_PIN_JK` and `SW2_PIN_JK` on falling edge triggers with priority level 10.

```
static inline void MCU_JK_init(void);
```